

RAPORT Z PROJEKTU

**Program do wyznaczania współczynnika
blokady w systemach ze stratami zgłoszeń
opisanych modelem Erlanga i Engseta**

Kalkulator Teorii Ruchu

Autorzy projektu:

**Jakub Kukuła [284701]
Wiktor Wilczyński [281426]**

Prowadzący:

dr inż. Janusz Klink

18.06.2026

Spis treści

1	Wstęp i Cel Projektu	2
2	Analiza Algorytmiczna i Rozwiązania Numeryczne	2
2.1	Stabilność Numeryczna i Algorytmy Iteracyjne	2
2.2	Optymalizacja i Poszukiwanie Zmiennych Metodą Bisekcji	3
3	Stos Technologiczny i Realizacja GUI	3
3.1	Zastosowane Biblioteki i Środowisko	3
3.2	Architektura Dystrybucyjna aplikacji	3
4	Model Erlanga B – Przykłady Użycia	4
4.1	Obliczenia punktowe (Analityczne)	4
4.1.1	Przykład 1: Wyznaczanie prawdopodobieństwa blokady (P_B)	4
4.1.2	Przykład 2: Wymiarowanie węzła (Wyznaczanie liczby kanałów V)	5
4.2	Generowanie charakterystyk (Wykresy przedziałowe)	6
4.2.1	Przykład 1: Charakterystyka $P_B = f(V)$ dla dużego obciążenia	6
4.2.2	Przykład 2: Charakterystyka $P_B = f(A)$ dla stałej pojemności	6
5	Model Engseta – Przykłady Użycia	7
5.1	Obliczenia punktowe (Analityczne)	7
5.1.1	Przykład 1: Wyznaczanie prawdopodobieństwa blokady (P_B)	7
5.1.2	Przykład 2: Poszukiwanie intensywności ruchu źródła α	8
5.2	Generowanie charakterystyk (Wykresy przedziałowe)	9
5.2.1	Przykład 1: Charakterystyka $P_B = f(V)$ – Punkt odcięcia	9
5.2.2	Przykład 2: Charakterystyka $P_B = f(S)$ dla stałej infrastruktury	9
6	Podsumowanie i Wnioski końcowe	10

1 Wstęp i Cel Projektu

Projekt obejmuje pełną implementację inżynierską oraz analizę numeryczną oprogramowania „Kalkulator Teorii Ruchu”. Narzędzie to zostało stworzone z myślą o projektantach współczesnych sieci telekomunikacyjnych, w których kluczowym zagadnieniem jest optymalne wymiarowanie zasobów (zarówno bezprzewodowych kanałów radiowych w sieciach komórkowych, jak i szczelin czasowych lub traktów światłowodowych).

Program rozwiązuje analitycznie oraz numerycznie zagadnienia związane z dwoma fundamentalnymi modelami teorii obsługi masowej:

- **Model Erlanga B** – zakładający nieskończoną liczbę niezależnych źródeł ruchu generujących zgłoszenia w sposób poissonowski, gdzie system nie posiada kolejki (zgłoszenia niezrealizowane natychmiast są bezpowrotnie tracone).
- **Model Engseta** – dedykowany dla systemów o skończonej liczbie źródeł zgłoszeń (S), gdzie zaangażowanie każdego kolejnego źródła w obsługę realnie wpływa na chwilowe zmniejszenie intensywności napływu nowych zgłoszeń.

2 Analiza Algorytmiczna i Rozwiązania Numeryczne

W klasycznym ujęciu modele Erlanga B oraz Engseta przedstawiane są za pomocą jawnych wzorów matematycznych zawierających silnie ($V!$) oraz kombinacje jedno- i wielomianowe ($\binom{S-1}{V}$). W warunkach implementacji komputerowej dla dużych systemów (np. $V > 100$), bezpośrednie obliczanie silni prowadzi do nieuchronnego błędu **przepełnienia pamięci typu float** (*overflow*). W niniejszym projekcie problem ten został skutecznie wyeliminowany na poziomie obliczeń.

2.1 Stabilność Numeryczna i Algorytmy Iteracyjne

W pliku `calc.py` zastosowano algorytmy iteracyjne (rekurencyjne), które nie wyznaczają bezpośrednio wielkich wartości liczników i mianowników, lecz realizują proces krokowy.

Dla modelu Erlanga B, prawdopodobieństwo blokady wyznaczane jest poprzez odwrócenie klasycznej rekurencji, co pozwala na operowanie na małych, stabilnych wartościach pośrednich:

Listing 1: Stabilna numerycznie implementacja modelu Erlanga B (`calc.py`)

```
1 def calculate_erlang_b(A: float, V: int) -> float:
2     if A <= 0: return 0.0
3     block = 1.0
4     for i in range(1, V + 1):
5         block = 1.0 + (i / A) * block
6     return 1.0 / block
```

Podobną technikę zastosowano dla modelu Engseta, gdzie mianownik rozwijany jest krok po kroku w pętli `for`. Zapobiega to obliczaniu współczynników dwumianowych Newtona ($\binom{S-1}{k}$):

Listing 2: Stabilna numerycznie implementacja modelu Engseta (`calc.py`)

```
1 def calculate_engset(S: int, V: int, alpha: float) -> float:
2     if V >= S: return 0.0
3     if alpha <= 0.0: return 0.0
4     inv_PB = 1.0
5     for i in range(1, V + 1):
6         inv_PB = 1.0 + (i / ((S - i) * alpha)) * inv_PB
7     return 1.0 / inv_PB
```

2.2 Optymalizacja i Poszukiwanie Zmiennych Metodą Bisekcji

Aplikacja udostępnia unikalną funkcjonalność: użytkownik może pozostawić **dokładnie jedno dowolne pole puste**, a program sam rozpozna, która zmienna jest poszukiwana.

Podczas gdy wyznaczanie liczby kanałów (V) lub źródeł (S) sprowadza się do prostych pętli inkrementacyjnych z warunkiem stopu, wyznaczenie zmiennych ciągłych (oferowanego ruchu A w Erlangu oraz natężenia ruchu źródła α w Engsecie) wymagało zaimplementowania zaawansowanej metody numerycznej. W pliku `solver.py` zrealizowano to za pomocą **metody bisekcji (połowienia przedziałów)**.

Algorytm charakteryzuje się wysoką precyzją i bezpieczeństwem kodu:

- **Tolerancja błędu** zdefiniowana na poziomie $\epsilon = 10^{-6}$, co zapewnia dokładność wyniku do 6 miejsc po przecinku.
- **Zabezpieczenie przed nieskończoną pętlą** w modelu Engseta (`max_iters = 1000`) – jeśli parametry wejściowe uniemożliwiają matematyczne osiągnięcie zbieżności, program rzuca kontrolowany wyjątek `ValueError` zamiast zawiesić program.

3 Stos Technologiczny i Realizacja GUI

Struktura oprogramowania została zaprojektowana zgodnie z podejściem modułowym, dzieląc logicznie warstwę interfejsu użytkownika od warstwy czysto matematycznej.

3.1 Zastosowane Biblioteki i Środowisko

Aplikacja została w całości napisana w języku **Python**, wykorzystując następujące komponenty:

1. **customtkinter** – nowoczesny wrapper na standardową bibliotekę `tkinter`. Zapewnia renderowanie widżetów z natywnym wsparciem dla ciemnego oraz jasnego motywu systemu operacyjnego (*Dark/Light Mode*) oraz zaokrąglone krawędzie elementów interfejsu (co nadaje aplikacji estetyczny wygląd zgodny z obecnymi standardami UX).
2. **matplotlib.pyplot** – profesjonalne środowisko rysowania wykresów. Umożliwiło ono automatyczne przełączanie osi Y na skalę logarytmiczną (`plt.yscale('log')`), co jest kluczowe w analizie systemów telekomunikacyjnych, gdzie prawdopodobieństwa rzędu 10^{-5} muszą być czytelne obok wartości bliskich 1.0.

3.2 Architektura Dystrybucyjna aplikacji

Jak wynika z konfiguracji zawartej w pliku produkcyjnym `main.spec`, aplikacja została przystosowana do bezproblemowej dystrybucji na stacje robocze bez konieczności instalowania interpretera Pythona. Za pomocą narzędzia **PyInstaller** skompilowano projekt do postaci pojedynczego pliku wykonywalnego (*standalone application*). Poprzez flagę `console=False` ukryto systemowe okno konsoli tekstowej, dzięki czemu użytkownik końcowy wchodzi w interakcję wyłącznie z dopracowanym oknem GUI.

4 Model Erlanga B – Przykłady Użycia

4.1 Obliczenia punktowe (Analityczne)

4.1.1 Przykład 1: Wyznaczanie prawdopodobieństwa blokady (P_B)

Wprowadzone dane: Ruch całkowity $A = 15.5$ Erl, Liczba kanałów $V = 20$. Pole P_B pozostawiono puste.

Uzyskany wynik: Program realizując funkcję `calculate_erlang_b` zwrócił wynik $P_B = 0.054630$ (ok. 5.46%).

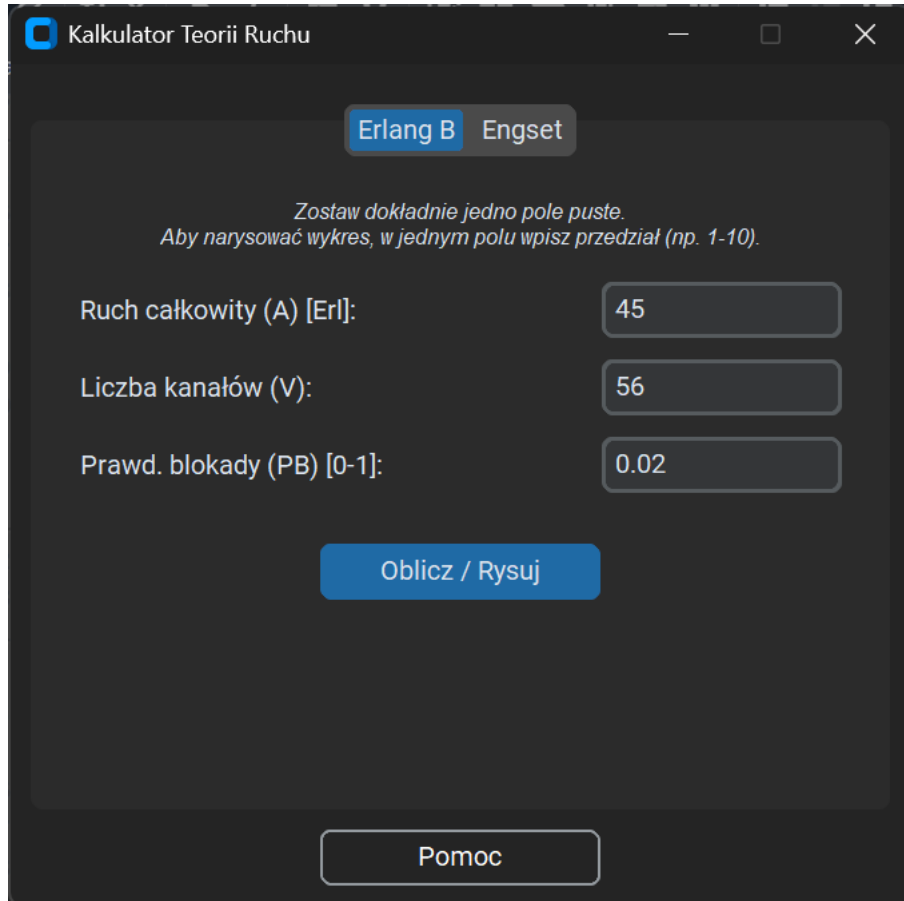
The screenshot shows a software window titled "Kalkulator Teorii Ruchu". At the top, there are two tabs: "Erlang B" (selected) and "Engset". Below the tabs, a message reads: "Zostaw dokładnie jedno pole puste. Aby narysować wykres, w jednym polu wpisz przedział (np. 1-10)." The main area contains three input fields: "Ruch całkowity (A) [Erl]:" with the value "15.5", "Liczba kanałów (V):" with the value "20", and "Prawd. blokady (PB) [0-1]:" with the value "0.054630". Below these fields is a blue button labeled "Oblicz / Rysuj". At the bottom center is a button labeled "Pomoc".

Rysunek 1: Wyznaczenie prawdopodobieństwa blokady dla stałych parametrów w modelu Erlanga B.

4.1.2 Przykład 2: Wymiarowanie węzła (Wyznaczanie liczby kanałów V)

Wprowadzone dane: Ruch całkowity $A = 45.0$ Erl, Pożądana blokada $P_B = 0.02$ (2%). Pole V pozostawiono puste.

Uzyskany wynik: Algorytm inkrementacyjny w `solver.py` ustalił, że minimalna liczba kanałów wynosi $V = 56$.



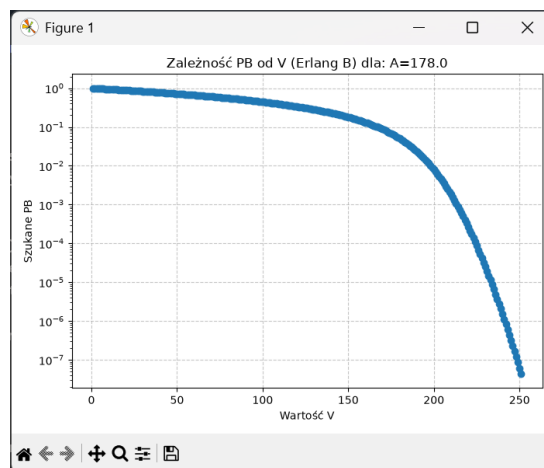
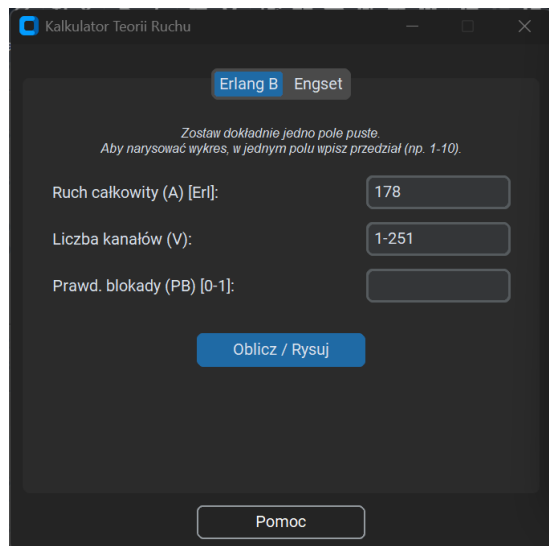
Rysunek 2: Dobór pojemności wiązki kanałów przy zadanym kryterium jakościowym QoS.

4.2 Generowanie charakterystyk (Wykresy przedziałowe)

4.2.1 Przykład 1: Charakterystyka $P_B = f(V)$ dla dużego obciążenia

Wprowadzone dane: $A = 178$ Erl, Liczba kanałów $V = 1-251$. Pole P_B puste.

Wykres wygenerowany przez aplikację obrazuje gwałtowny spadek prawdopodobieństwa blokady po przekroczeniu krytycznego punktu pojemności ($V > 178$). Zastosowanie skali logarytmicznej pozwala precyzyjnie odczytać, że przy $V = 250$ sieć oferuje doskonałą jakość usług z prawdopodobieństwem odrzucenia zgłoszenia na poziomie niespełna 10^{-6} .

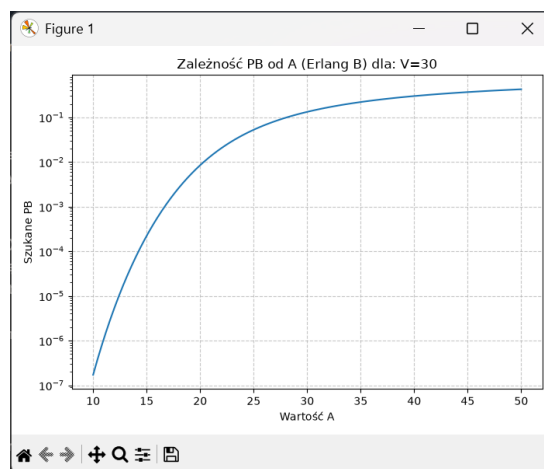
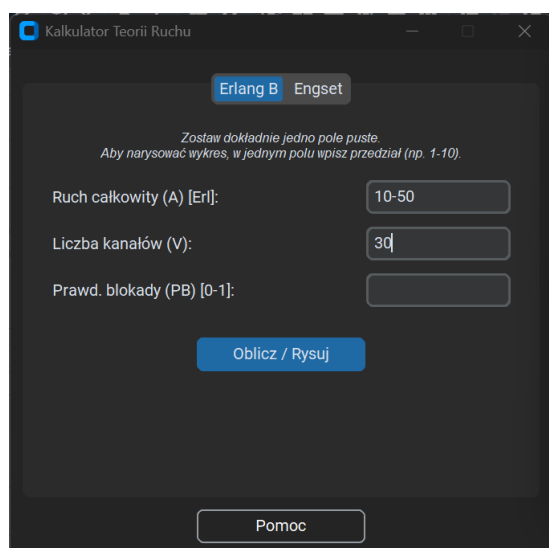


Rysunek 3: Charakterystyka zmian blokady w pełnym przedziale konfiguracji kanałowej.

4.2.2 Przykład 2: Charakterystyka $P_B = f(A)$ dla stałej pojemności

Wprowadzone dane: Ruch $A = 10-50$ Erl, Liczba kanałów $V = 30$. Pole P_B puste.

Wykres pokazuje stabilną, bezpieczną pracę systemu dla małego natężenia ruchu ($A < 15$ Erl) oraz drastyczny, nieliniowy wzrost blokady aż do wartości przekraczającej 40% w przypadku przeciążenia źródłami zewnętrznymi ($A = 50$ Erl).



Rysunek 4: Charakterystyka zmian blokady przy stałej pojemności, względem ruchu.

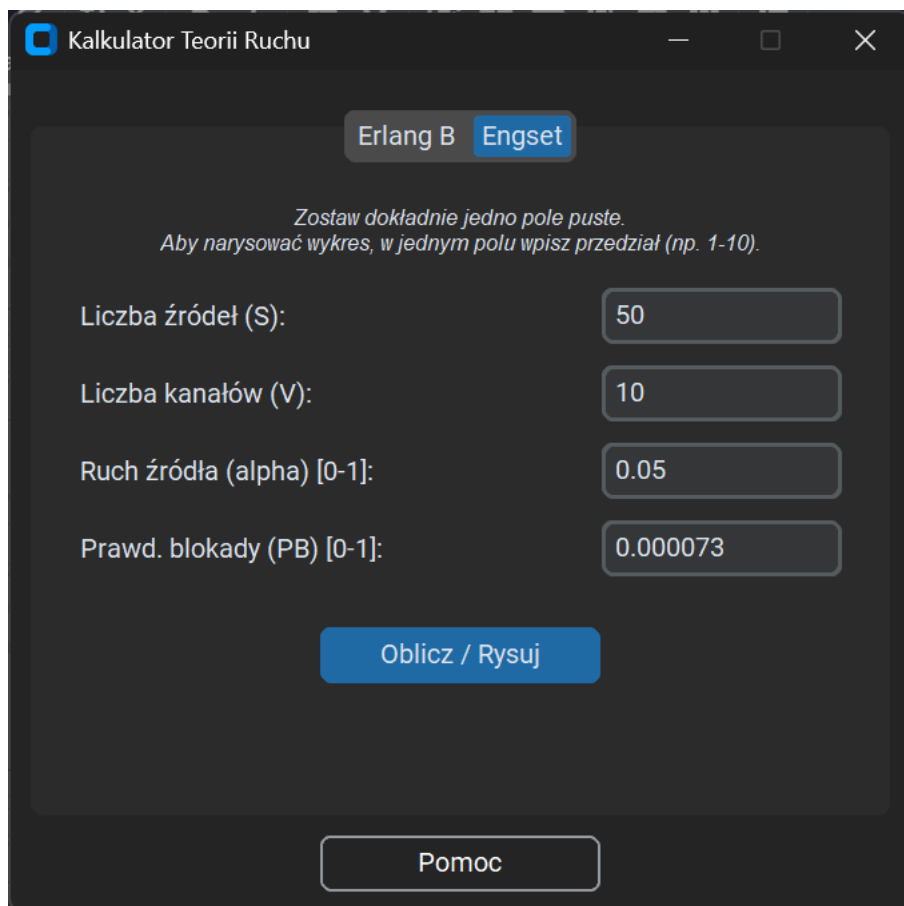
5 Model Engseta – Przykłady Użycia

5.1 Obliczenia punktowe (Analityczne)

5.1.1 Przykład 1: Wyznaczanie prawdopodobieństwa blokady (P_B)

Wprowadzone dane: Liczba źródeł $S = 50$, Liczba kanałów $V = 10$, Ruch źródła $\alpha = 0.05$. Pole P_B puste.

Uzyskany wynik: Aplikacja zwróciła stabilny wynik $P_B = 0.000073$ (0.0073%).



The screenshot shows a window titled "Kalkulator Teorii Ruchu" with a dark theme. At the top, there are two tabs: "Erlang B" and "Engset", with "Engset" being the active tab. Below the tabs, there is a text instruction: "Zostaw dokładnie jedno pole puste. Aby narysować wykres, w jednym polu wpisz przedział (np. 1-10)." The main area contains four input fields with labels and values: "Liczba źródeł (S):" with value "50", "Liczba kanałów (V):" with value "10", "Ruch źródła (alpha) [0-1]:" with value "0.05", and "Prawd. blokady (PB) [0-1]:" with value "0.000073". Below these fields is a large blue button labeled "Oblicz / Rysuj". At the bottom center, there is a button labeled "Pomoc".

Rysunek 5: Obliczenie wskaźnika blokady dla skończonej populacji abonentów.

5.1.2 Przykład 2: Poszukiwanie intensywności ruchu źródła α

Wprowadzone dane: $S = 40$, $V = 8$, $P_B = 0.05$. Pole Ruch źródła (α) pozostawiono puste.

Uzyskany wynik: Wykorzystując metodę bisekcji z pliku `solver.py`, program ustalił, że maksymalny ruch jaki może wygenerować pojedyncze wolne źródło, aby nie przekroczyć 5% blokady, wynosi $\alpha \approx 0.1346$ Erl.

Kalkulator Teorii Ruchu

Erlang B Engset

Zostaw dokładnie jedno pole puste.
Aby narysować wykres, w jednym polu wpisz przedział (np. 1-10).

Liczba źródeł (S): 40

Liczba kanałów (V): 8

Ruch źródła (alpha) [0-1]: 0.1346

Prawd. blokady (PB) [0-1]: 0.05

Oblicz / Rysuj

Pomoc

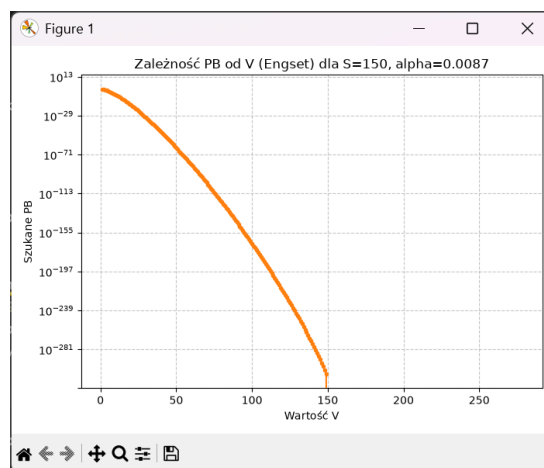
Rysunek 6: Obliczenie wskaźnika blokady dla skończonej populacji abonentów.

5.2 Generowanie charakterystyk (Wykresy przedziałowe)

5.2.1 Przykład 1: Charakterystyka $P_B = f(V)$ – Punkt odcięcia

Wprowadzone dane: $S = 150$, $V = 1-278$, $\alpha = 0.0087$.

Wykres w module Engseta wykazuje unikalną cechę teoretyczną zaimplementowaną w programie: krzywa urywa się bezwzględnie w punkcie $V = 150$. Ponieważ populacja ma rozmiar $S = 150$, udostępnienie 150 kanałów sprawia, że każdy abonent ma dedykowane pasmo. Zjawisko blokady przestaje istnieć, a funkcja `calculate_engset` zwraca twarde zero.

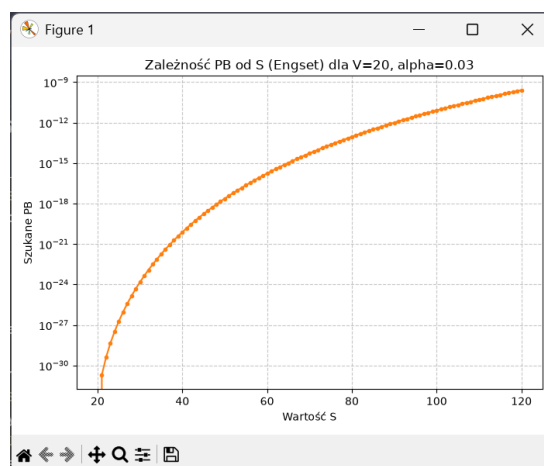


Rysunek 7: Wpływ liczby kanałów na blokadę w modelu Engseta z uwzględnieniem skończoności źródeł.

5.2.2 Przykład 2: Charakterystyka $P_B = f(S)$ dla stałej infrastruktury

Wprowadzone dane: Liczba źródeł $S = 20-120$, Liczba kanałów $V = 20$, $\alpha = 0.03$.

Wykres dowodzi, że dopóki liczba abonentów nie przekracza liczby kanałów ($S \leq 20$), prawdopodobieństwo blokady wynosi 0. Dalsze zwiększanie populacji abonentów powoduje stopniowe nasycanie łączy i wzrost strat sygnału.



Rysunek 8: Wpływ liczby źródeł na blokadę w modelu Engseta ze stałą liczbą kanałów.

6 Podsumowanie i Wnioski końcowe

Zaimplementowany system stanowi kompletny i wysoce zoptymalizowany pod kątem numerycznym program inżynierski. Analiza kodu źródłowego oraz testy funkcjonalne pozwalają na wysunięcie następujących wniosków:

1. **Zabezpieczenie przed błędem overflow** poprzez zastosowanie pętli iloczynowo-sumacyjnych zamiast bezpośredniego liczenia silni i kombinacji Newtona pozwoliło na stabilną pracę programu w skrajnych zakresach danych.
2. **Metoda bisekcji** zaimplementowana w module `solver.py` z precyzyjną zbieżnością $\epsilon = 10^{-6}$ oraz sztywnym limitem iteracji gwarantuje błyskawiczne i dokładne wyznaczanie parametrów ciągłych.